Calendar Day View with Overlapping Events

- **Problem:** Build a Google-Calendar-style day view. Events sit on a 24-hour vertical grid; events that overlap in time must be laid out side by side without covering each other.
- **Tech stack:** React (Vite, JavaScript) *(default, none given)*
- **Time limit:** 60 minutes *(default, none given)*
- **Often asked at:** Google, Meta, Uber. **Why it matters:** it is a layout *algorithm* problem disguised as a UI problem.

PART A: Thought process

General steps for any machine coding task

1. **Clarify requirements**: what must work, what is out of scope. Write assumptions down.
2. **List interactions**: every user action and its expected result. This is your test checklist.
3. **Sketch components**: a small tree, one responsibility each.
4. **Design state**: minimal source of truth; derive everything else.
5. **Define data contracts**: data shape, API/mock shape, function signatures.
6. **Find the hard parts**: solve them on paper before typing.
7. **Build in layers**: a working skeleton first, then one feature at a time, keeping it runnable.
8. **Handle states and edge cases**: loading, empty, error, invalid input.
9. **Wrap up**: walk the checklist, explain trade-offs, list what you'd add.

Time budget for 60 minutes

Phase	Minutes
Requirements, questions, assumptions	5
Interactions, components, state, contracts	7
Overlap layout algorithm (pure function + quick checks)	15
Grid + event rendering (positioning)	13
Add / delete event, validation	8
Loading / error / empty states, styling	7
Walk-through, edge cases, trade-offs	5

Think out loud

Say every decision and its reason as you make it: *"I'll store time as minutes since midnight because the layout math becomes plain subtraction."* The interviewer grades your reasoning as much as the result. If you are stuck, say what you are trying; silence looks like being lost.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show one day as a vertical timeline, 00:00 to 24:00, with hour labels and hour lines.
- Render each event as a block: **top** = start time, **height** = duration.
- **Overlapping events sit side by side** and share the width; no block covers another.
- Events that do not overlap use the full width.
- An event can stretch into empty columns to its right (like Google Calendar).
- Add an event (title, start, end) and delete an event; the layout recalculates.
- Load the day's events from an API (mocked).

Non-functional requirements

Area	What it means here
Performance	Layout is `O(n log n)` sort + linear grouping; recompute only when events change (`useMemo`).
Scalability	Works for a typical day (tens of events); algorithm stays correct for hundreds.
Error handling	API failure shows an error with Retry; invalid events (end ≤ start) are dropped or rejected.
Accessibility	Every event has an `aria-label` with title and time; delete is a real `<button>` ; form errors use `role="alert"` .
Reusability	The layout is a **pure function** usable by week view, resource view, any timeline.
Extensibility	Drag to move/resize, multi-day, time zones can be added without touching the layout core.

Clarifying questions to ask the interviewer

1. Is it a single day only, or do events span multiple days / cross midnight?
2. When events overlap, should they share width equally per *overlap group*, or should non-conflicting events expand to fill free space?
3. Do events that touch (10:00–11:00 and 11:00–12:00) count as overlapping?
4. Do we need to add/edit/delete events, or only display them?
5. What does the data look like: ISO timestamps, "HH:MM" strings, or minutes? Time zones?
6. Do we need all-day events and a current-time indicator?

Assumptions we build with

- Single day, 24 hours; an event's `start` and `end` are on the same day.
- Time is stored as **minutes since midnight** (0–1440).
- Touching events do **not** overlap (`a.start < b.end && b.start < a.end`).
- Width is shared equally inside one **overlap group** (cluster); an event may **expand right** into columns where nothing overlaps it.
- Add + delete are in scope; drag/resize, all-day events and recurring events are out of scope.

- The API is mocked with a 600 ms delay.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	"Loading events…", then grid renders, scrolled to 8 AM
API fails	Error message with **Retry** button
API returns no events	Empty message + empty grid
Event 9:00–9:30 alone	Full-width block, top = 540px, height = 30px
Two events overlap	Each takes 50% width, side by side
A overlaps B, B overlaps C, A does not overlap C	All three in one group with 2 columns; C reuses A's column
Events touch (10–11, 11–12)	Both full width (no overlap)
Event has free columns on its right	It stretches over them (`span > 1`)
Add event with valid times	Block appears; layout of its group recalculates
Add with empty title	"Title is required"
Add with end ≤ start	"End time must be after start time"
Click × on an event	Event removed; neighbours re-expand
Very short event (5 min)	Rendered with a minimum height so it stays clickable
Bad row from API (end < start)	Silently dropped during normalization

---

3. Design the architecture

Module	Responsibility
`App`	Owns `events` + load status; fetch, add, delete
`EventForm`	Controlled form, validation, emits a new event
`DayView`	Hour grid, scroll container, calls `layoutEvents`
`EventBlock`	Turns `{start, end, col, cols, span}` into absolute CSS position
`layoutEvents()`	**Pure** overlap algorithm: events → events with `col / cols / span`
`time.js`	`toMinutes("09:30")` , `formatTime(570)`
`eventsApi.js`	Mock fetch + normalize API shape to UI shape

```
App  (events, status)
├── EventForm        → onAdd(event)
└── DayView          (events) ── useMemo(layoutEvents)
    ├── hour labels + hour lines
    └── EventBlock × n  → onDelete(id)

utils/layoutEvents.js   (pure, no React)
```

```
utils/time.js
services/eventsApi.js
```

Start simple: one `App` + one `DayView`. The only thing worth splitting early is the **layout algorithm**, because it is the part you need to test in isolation.

---

4. Design the state / data model

State or field	Purpose
`events`	Source of truth: `[{ id, title, start, end }]` (minutes)
`status`	`'loading' | 'success' | 'error'` for the fetch
`error`	Error message to show
`form` (in `EventForm`)	Controlled inputs `{ title, start, end }`
`error` (in `EventForm`)	Validation message

```
const [events, setEvents] = useState([]);
const [status, setStatus] = useState('loading'); // 'loading' | 'success' | 'error'
const [error, setError] = useState('');

// derived, never stored
const positioned = useMemo(() => layoutEvents(events), [events]);
```

Senior note:

- `col`, `cols`, `span`, `top`, `height`, `left`, `width` are all **derived**. Storing them would mean every add/delete has to patch the layout of neighbours by hand: a bug factory. Recompute from `events` instead.
- `App` owns `events` because both the form (add) and the grid (delete) change it.
- `EventForm` owns its own draft state: nobody else needs half-typed input.
- Store times as **numbers (minutes)**, not strings or `Date`s: comparisons and pixel math become trivial and time-zone free.

---

5. Design the data structures and contracts

Example data

```
// API shape (what the server sends)
{ id: 'e2', title: 'Design review', start: '09:15', end: '10:30' }

// UI shape (after normalize)
{ id: 'e2', title: 'Design review', start: 555, end: 630 }

// After layoutEvents
{ id: 'e2', title: 'Design review', start: 555, end: 630, col: 1, cols: 2, span: 1 }
```

Stable `id`s matter: they are the React `key`, the delete target, and they keep a block's DOM node stable when the layout shifts. New events use `crypto.randomUUID()`.

Contracts

Contract	Input	Output
`GET /events?date=2026-11-02` (mocked by `fetchEvents()`)	—	`Promise<Event[]>` (minutes), rejects with `Error`
`layoutEvents(events)`	`Event[]` in any order	`PositionedEvent[]` with `col`, `cols`, `span`
`EventBlock` props	`event`, `pxPerMin`, `onDelete(id)`	absolutely-positioned `<article>`
`EventForm` props	`onAdd(event)`	calls `onAdd({ id, title, start, end })` only when valid

CSS from the layout result:

```
top    = start * pxPerMin
height = max((end - start) * pxPerMin, MIN_HEIGHT)
left   = col / cols * 100%
width  = span / cols * 100% - gap
```

---

6. Trace the data flow

Adding "Hotfix" 9:45–10:15 into the busy morning:

```
User fills form and presses "Add event"
        |
        ▼
EventForm.handleSubmit → validate (title, end > start)
        |  toMinutes("09:45") = 585, toMinutes("10:15") = 615
        ▼
onAdd({ id: uuid, title: 'Hotfix', start: 585, end: 615 })
        |
        ▼
App: setEvents(prev => [...prev, ev])        ← new array reference
        |
        ▼
DayView re-renders → useMemo sees new `events` → layoutEvents()
        |  sort → group 9:00–11:00 now needs 3 columns
        |  Hotfix → col 2, cols 3; Standup/Design/1:1/Code review → cols 3
        ▼
EventBlock × n get new col/cols/span → new left/width
        |
        ▼
Morning group re-renders at 1/3 width each; other groups unchanged
```

---

7. Important technical challenges

Challenge 1: Splitting events into overlap groups (clusters)

Problem: Width must be shared only among events that actually conflict. Standup (9:00) and Lunch (12:30) must not shrink each other. But overlap is *transitive* for layout: A 9:00–10:00, B 9:30–10:30, C 10:00–11:00. A and C don't overlap, yet they must share B's column count, otherwise C draws full width over B.

Solution: Sort by start time and sweep. Keep the **latest end** of the current group. If the next event starts at or after that end, nothing open can touch it: close the group and start a new one.

```
const sorted = [...events].sort((a, b) => a.start - b.start || b.end - a.end);
let groupEnd = -1;
for (const ev of sorted) {
  if (ev.start >= groupEnd) closeGroup();     // independent new group
  // ...place ev in a column...
  groupEnd = Math.max(groupEnd, ev.end);
}
closeGroup();
```

Challenge 2: Assigning columns with reuse (greedy first-fit)

Problem: Giving each overlapping event its own new column wastes space. In the chain above, C can reuse A's column once A ends, so the group needs 2 columns, not 3.

Solution: Track the end time of the last event in each column. Put the event into the **first column that is free** at its start; open a new column only if none is free. Because events are processed in start order, this greedy choice uses the minimum number of columns (the classic interval-partitioning result).

```
let col = columnEnds.findIndex((end) => end <= ev.start);
if (col === -1) {
  col = columnEnds.length;
  columnEnds.push(ev.end);
} else {
  columnEnds[col] = ev.end;
}
group.push({ ...ev, col });
// on closeGroup: every event in the group gets cols = columnEnds.length
```

Tie-break `b.end - a.end`: on equal starts the **longer** event goes first and gets the left column, which looks cleaner.

Challenge 3: Expanding into free space (span)

Problem: In a 3-column group, "Bug bash" 15:00–15:45 sits in column 1 but nothing in column 2 overlaps it. Equal-width columns leave an ugly empty strip.

Solution: When closing a group, for each event walk right column by column while no event in that column overlaps it.

```
let span = 1;
while (
```

```
  ev.col + span < cols &&
  !group.some((o) => o.col === ev.col + span && overlaps(o, ev))
) {
  span++;
}
```

`width = span / cols`. This is `O(g²)` per group of size `g`, which is fine because groups are small (a few events).

Challenge 4: Turning minutes into pixels

Problem: The layout returns abstract numbers; the grid needs exact pixels, and a 5-minute event would be 5px tall and unclickable.

Solution: One constant `HOUR_HEIGHT = 60` → `1px per minute`. Absolutely position blocks inside a `position: relative` column; percentages for horizontal, pixels for vertical; clamp height to a minimum.

```
const style = {
  top: start * pxPerMin,
  height: Math.max((end - start) * pxPerMin, MIN_HEIGHT),
  left: `${(col / cols) * 100}%`,
  width: `calc(${(span / cols) * 100}% - ${GAP}px)`,
};
```

---

8. Implement in the correct order

#	Step	Working result
1	Vite scaffold, `App` with hard-coded events array	Something renders
2	`time.js` (`toMinutes` , `formatTime`)	Times convert both ways
3	`DayView` grid: 24 hour labels + lines, scroll container	Empty calendar visible
4	`EventBlock` with `top/height` , full width	Events at the right times (overlaps collide)
5	`layoutEvents` v1: grouping + columns	Overlaps sit side by side
6	Node assert checks for the algorithm	Algorithm proven on chains, touches, reuse
7	`span` expansion	No wasted empty strips
8	`eventsApi` mock + loading / error / empty states	Real data flow with Retry
9	`EventForm` with validation, `addEvent`	Add events, layout recalculates
10	Delete button, `deleteEvent`	Remove events, neighbours re-expand
11	Scroll to 8 AM, min height, aria labels, styling	Polished and accessible

---

9. Cover all states and edge cases

State / edge case	Handling
Initial / loading	`status = 'loading'` → "Loading events…"
Success	Grid with positioned events, scrolled to 8 AM
Empty	"No events today. Add one above." + empty grid
Error	Message + Retry (calls `load` again)
Touching events	Strict `<` in `overlaps` → no shared columns
Identical start and end	Both in the same group, 2 columns
Same start, different length	Longer one takes the left column
Chain overlap (A–B–C)	One group via `groupEnd`; C reuses A's column
Event fully inside another	Two columns; the inner one may span if nothing else blocks
Very short event	`MIN_HEIGHT` 20px (visual only, layout uses real times)
end ≤ start from API	Dropped in `normalize`
end ≤ start from form	"End time must be after start time"
Empty / whitespace title	"Title is required"
Events unsorted from API	`layoutEvents` sorts a copy; input never mutated
Long title	Ellipsis; full label in `title` + `aria-label`
Many events in one slot (10+)	Still correct; blocks get narrow (see trade-offs)

---

10. Final structure

```
_61_calendar_day_view_overlapping_events/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx                  # events + status, add / delete
    ├── App.css
    ├── components/
    │   ├── DayView.jsx          # grid + useMemo(layoutEvents)
    │   ├── EventBlock.jsx       # absolute positioning of one event
    │   └── EventForm.jsx        # controlled form + validation
    ├── services/
    │   └── eventsApi.js         # mock fetch + normalize
    └── utils/
        ├── layoutEvents.js      # overlap layout algorithm (pure)
        ├── layoutEvents.test.js # node assert checks
        └── time.js              # toMinutes / formatTime
```

---

11. Full working code

`src/utils/time.js`

```
// "09:30" -> 570 (minutes since midnight)
export function toMinutes(hhmm) {
  const [h, m] = hhmm.split(':').map(Number);
  return h * 60 + m;
}

// 570 -> "9:30 AM"
export function formatTime(minutes) {
  const h = Math.floor(minutes / 60) % 24;
  const m = String(minutes % 60).padStart(2, '0');
  const suffix = h < 12 ? 'AM' : 'PM';
  return `${h % 12 || 12}:${m} ${suffix}`;
}
```

`src/utils/layoutEvents.js`

```
// Touching events (10:00-11:00 and 11:00-12:00) do NOT overlap.
const overlaps = (a, b) => a.start < b.end && b.start < a.end;

/**
 * Input:  [{ id, title, start, end }]  (start/end in minutes)
 * Output: same events + { col, cols, span }
 *   col  = column index inside its overlap group
 *   cols = number of columns in that group (width = 1 / cols)
 *   span = how many columns it can stretch over to the right
 */
export function layoutEvents(events) {
  // Earlier start first; on a tie the longer event goes first (takes the left column).
  const sorted = [...events].sort((a, b) => a.start - b.start || b.end - a.end);

  const result = [];
  let group = []; // events of the current overlap group
  let columnEnds = []; // columnEnds[i] = end of the last event placed in column i
  let groupEnd = -1; // latest end in the group

  const closeGroup = () => {
    const cols = columnEnds.length;
    for (const ev of group) {
      let span = 1;
      // stretch right while the next column has nothing overlapping this event
      while (
        ev.col + span < cols &&
        !group.some((o) => o.col === ev.col + span && overlaps(o, ev))
      ) {
        span++;
      }
      result.push({ ...ev, cols, span });
    }
    group = [];
    columnEnds = [];
  };
```

```
  for (const ev of sorted) {
    // starts after everything in the group ended -> a new, independent group
    if (ev.start >= groupEnd) closeGroup();

    // first column that is free at ev.start, else open a new column
    let col = columnEnds.findIndex((end) => end <= ev.start);
    if (col === -1) {
      col = columnEnds.length;
      columnEnds.push(ev.end);
    } else {
      columnEnds[col] = ev.end;
    }

    group.push({ ...ev, col });
    groupEnd = Math.max(groupEnd, ev.end);
  }
  closeGroup();

  return result;
}
```

src/utils/layoutEvents.test.js

```
// Run: node src/utils/layoutEvents.test.js
import assert from 'node:assert/strict';
import { layoutEvents } from './layoutEvents.js';

const byId = (list) => Object.fromEntries(layoutEvents(list).map((e) => [e.id, e]));
const ev = (id, start, end) => ({ id, title: id, start, end });

// 1. No overlap -> full width each
let r = byId([ev('a', 60, 120), ev('b', 180, 240)]);
assert.deepEqual([r.a.col, r.a.cols, r.b.col, r.b.cols], [0, 1, 0, 1]);

// 2. Touching events do not overlap
r = byId([ev('a', 600, 660), ev('b', 660, 720)]);
assert.equal(r.a.cols, 1);
assert.equal(r.b.cols, 1);

// 3. Two overlapping -> half width each
r = byId([ev('a', 540, 600), ev('b', 570, 630)]);
assert.deepEqual([r.a.col, r.b.col, r.a.cols], [0, 1, 2]);

// 4. Column reuse: c fits under a once a ends
r = byId([ev('a', 540, 600), ev('b', 540, 660), ev('c', 600, 630)]);
assert.equal(r.b.col, 0); // longer one takes the left column
assert.equal(r.a.col, 1);
assert.equal(r.c.col, 1);
assert.equal(r.c.cols, 2);

// 5. Chain: a overlaps b, b overlaps c, a does not overlap c -> one group of 2 columns
r = byId([ev('a', 0, 60), ev('b', 30, 90), ev('c', 60, 120)]);
assert.deepEqual([r.a.col, r.b.col, r.c.col, r.c.cols], [0, 1, 0, 2]);

// 6. Span: d can stretch over the empty column to its right
r = byId([ev('a', 0, 120), ev('b', 0, 30), ev('c', 0, 30), ev('d', 60, 90)]);
assert.equal(r.d.col, 1);
assert.equal(r.d.span, 2);
```

```
assert.equal(r.b.span, 1);

// 7. Empty input
assert.deepEqual(layoutEvents([]), []);

console.log('layoutEvents: all checks passed');
```

src/services/eventsApi.js

```
import { toMinutes } from '../utils/time';

// Mock API: GET /events?date=2026-11-02
const MOCK_EVENTS = [
  { id: 'e1', title: 'Standup', start: '09:00', end: '09:30' },
  { id: 'e2', title: 'Design review', start: '09:15', end: '10:30' },
  { id: 'e3', title: '1:1 with manager', start: '09:30', end: '10:00' },
  { id: 'e4', title: 'Code review', start: '10:00', end: '11:00' },
  { id: 'e5', title: 'Lunch', start: '12:30', end: '13:30' },
  { id: 'e6', title: 'Interview', start: '14:00', end: '15:00' },
  { id: 'e7', title: 'Sprint planning', start: '14:00', end: '16:00' },
  { id: 'e8', title: 'Coffee chat', start: '14:30', end: '15:00' },
  { id: 'e9', title: 'Bug bash', start: '15:00', end: '15:45' },
  { id: 'e10', title: 'Broken event', start: '17:00', end: '16:00' }, // invalid, dropped
];

// Convert API shape ("HH:MM") to UI shape (minutes) and drop invalid rows.
function normalize(raw) {
  return raw
    .map((e) => ({ id: e.id, title: e.title, start: toMinutes(e.start), end: toMinutes(e.end) }))
    .filter((e) => e.end > e.start);
}

export function fetchEvents({ fail = false } = {}) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (fail) reject(new Error('Could not load events'));
      else resolve(normalize(MOCK_EVENTS));
    }, 600);
  });
}
```

src/components/EventBlock.jsx

```
import { formatTime } from '../utils/time';

const GAP = 4; // px between side-by-side events
const MIN_HEIGHT = 20; // keep 5-minute events clickable

export default function EventBlock({ event, pxPerMin, onDelete }) {
  const { title, start, end, col, cols, span } = event;
  const label = `${title}, ${formatTime(start)} to ${formatTime(end)}`;

  const style = {
    top: start * pxPerMin,
    height: Math.max((end - start) * pxPerMin, MIN_HEIGHT),
    left: `${(col / cols) * 100}%`,
```

```
    width: `calc(${(span / cols) * 100}% - ${GAP}px)`,
  };

  return (
    <article className="event" style={style} aria-label={label} title={label}>
      <strong className="event-title">{title}</strong>
      <span className="event-time">
        {formatTime(start)} – {formatTime(end)}
      </span>
      <button
        type="button"
        className="event-delete"
        aria-label={`Delete ${title}`}
        onClick={() => onDelete(event.id)}
      >
        ×
      </button>
    </article>
  );
}
```

src/components/DayView.jsx

```
import { useEffect, useMemo, useRef } from 'react';
import { layoutEvents } from '../utils/layoutEvents';
import { formatTime } from '../utils/time';
import EventBlock from './EventBlock';

const HOUR_HEIGHT = 60; // px -> 1px per minute keeps the math trivial
const PX_PER_MIN = HOUR_HEIGHT / 60;
const HOURS = Array.from({ length: 24 }, (_, h) => h);

export default function DayView({ events, onDelete, scrollToHour = 8 }) {
  const scrollRef = useRef(null);

  // Layout is derived from events, never stored.
  const positioned = useMemo(() => layoutEvents(events), [events]);

  useEffect(() => {
    scrollRef.current.scrollTop = scrollToHour * HOUR_HEIGHT;
  }, [scrollToHour]);

  return (
    <div className="day-view" ref={scrollRef}>
      <div className="day-grid" style={{ height: 24 * HOUR_HEIGHT }}>
        <ol className="hour-labels" aria-hidden="true">
          {HOURS.map((h) => (
            <li key={h} style={{ height: HOUR_HEIGHT }}>
              {h === 0 ? '' : formatTime(h * 60).replace(':00', '')}
            </li>
          ))}
        </ol>

        <div className="events-column">
          {HOURS.map((h) => (
            <div key={h} className="hour-line" style={{ top: h * HOUR_HEIGHT }} />
          ))}
          {positioned.map((ev) => (
```

```
            <EventBlock key={ev.id} event={ev} pxPerMin={PX_PER_MIN} onDelete={onDelete} />
          ))}
        </div>
      </div>
    </div>
  );
}
```

`src/components/EventForm.jsx`

```
import { useState } from 'react';
import { toMinutes } from '../utils/time';

const EMPTY = { title: '', start: '09:00', end: '10:00' };

export default function EventForm({ onAdd }) {
  const [form, setForm] = useState(EMPTY);
  const [error, setError] = useState('');

  const update = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = (e) => {
    e.preventDefault();
    const title = form.title.trim();
    if (!title) return setError('Title is required');
    if (!form.start || !form.end) return setError('Start and end time are required');

    const start = toMinutes(form.start);
    const end = toMinutes(form.end);
    if (end <= start) return setError('End time must be after start time');

    onAdd({ id: crypto.randomUUID(), title, start, end });
    setForm(EMPTY);
    setError('');
  };

  return (
    <form className="event-form" onSubmit={handleSubmit} noValidate>
      <label>
        Title
        <input name="title" value={form.title} onChange={update} placeholder="Team sync" />
      </label>
      <label>
        Start
        <input type="time" name="start" value={form.start} onChange={update} />
      </label>
      <label>
        End
        <input type="time" name="end" value={form.end} onChange={update} />
      </label>
      <button type="submit">Add event</button>
      {error && (
        <p className="form-error" role="alert">
          {error}
        </p>
      )}
    </form>
```

```
  );
}
```

src/App.jsx

```
import { useCallback, useEffect, useState } from 'react';
import { fetchEvents } from './services/eventsApi';
import DayView from './components/DayView';
import EventForm from './components/EventForm';
import './App.css';

export default function App() {
  const [events, setEvents] = useState([]);
  const [status, setStatus] = useState('loading'); // 'loading' | 'success' | 'error'
  const [error, setError] = useState('');

  const load = useCallback(() => {
    setStatus('loading');
    fetchEvents()
      .then((data) => {
        setEvents(data);
        setStatus('success');
      })
      .catch((err) => {
        setError(err.message);
        setStatus('error');
      });
  }, []);

  useEffect(load, [load]);

  const addEvent = (ev) => setEvents((prev) => [...prev, ev]);
  const deleteEvent = (id) => setEvents((prev) => prev.filter((e) => e.id !== id));

  return (
    <main className="app">
      <header>
        <h1>Monday, 2 November</h1>
        <EventForm onAdd={addEvent} />
      </header>

      {status === 'loading' && <p className="status">Loading events…</p>}
      {status === 'error' && (
        <p className="status error" role="alert">
          {error} <button onClick={load}>Retry</button>
        </p>
      )}
      {status === 'success' && events.length === 0 && (
        <p className="status">No events today. Add one above.</p>
      )}
      {status === 'success' && <DayView events={events} onDelete={deleteEvent} />}
    </main>
  );
}
```

`src/App.css`

```
.app {
  max-width: 900px;
  margin: 0 auto;
  padding: 16px;
}

h1 {
  font-size: 1.4rem;
  margin: 0 0 12px;
}

.event-form {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  align-items: flex-end;
  margin-bottom: 16px;
}

.event-form label {
  display: flex;
  flex-direction: column;
  font-size: 0.8rem;
  gap: 4px;
}

.event-form input,
.event-form button {
  padding: 6px 8px;
  font: inherit;
}

.form-error,
.status.error {
  color: #c62828;
  width: 100%;
  margin: 0;
}

/* Scrollable 24h grid */
.day-view {
  height: 70vh;
  overflow-y: auto;
  border: 1px solid #ddd;
  border-radius: 8px;
}

.day-grid {
  display: flex;
  position: relative;
}

.hour-labels {
  list-style: none;
  margin: 0;
  padding: 0;
```

```
  width: 56px;
  flex-shrink: 0;
  font-size: 0.7rem;
  color: #666;
}

.hour-labels li {
  text-align: right;
  padding-right: 8px;
  transform: translateY(-0.5em); /* label sits on its line */
}

.events-column {
  position: relative;
  flex: 1;
  border-left: 1px solid #ddd;
}

.hour-line {
  position: absolute;
  left: 0;
  right: 0;
  border-top: 1px solid #eee;
}

.event {
  position: absolute;
  box-sizing: border-box;
  overflow: hidden;
  padding: 2px 20px 2px 6px;
  border-radius: 4px;
  border-left: 3px solid #1a73e8;
  background: #e8f0fe;
  font-size: 0.75rem;
  line-height: 1.2;
}

.event-title {
  display: block;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

.event-time {
  color: #555;
}

.event-delete {
  position: absolute;
  top: 2px;
  right: 2px;
  border: none;
  background: transparent;
  cursor: pointer;
  font-size: 0.9rem;
  line-height: 1;
}
```

```
.event-delete:focus-visible {
  outline: 2px solid #1a73e8;
}
```

`src/index.css`

```
body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #222;
  background: #fff;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

Run: `npm create vite@latest` (React, JavaScript), replace `src/` with the files above, `npm run dev`. Check the algorithm with `node src/utils/layoutEvents.test.js`.

---

12. Wrap-up

Edge cases handled

Touching events, chain overlaps, column reuse, nested events, identical times, unsorted input, invalid API rows, invalid form input, very short events, long titles, empty day, API failure with retry.

Trade-offs to mention

- **Equal width per group + span** (Google-style) vs. per-event *maximal* width: ours is simple and predictable; a fully optimal packing is harder and rarely worth it.
- **Recompute the whole layout on every change** vs. incremental update: `O(n log n)` for tens of events is microseconds; incremental adds complexity for no visible gain.
- **Minutes since midnight** keeps math simple but ignores time zones and DST; a real app converts from ISO + zone at the API boundary.
- **Absolute positioning** vs. CSS Grid (`grid-row: span N`): absolute positioning gives pixel-exact minute precision and easy overlap control.
- **Min height** can make tiny events visually touch the next one even though their real times don't overlap.

What to add with more time

- Drag to move / resize with 15-minute snapping (pointer events + optimistic update).
- Click empty slot to create an event at that time.
- Current-time red line updating every minute.
- Multi-day / week view reusing `layoutEvents` per day column; events crossing midnight split into two segments.
- Persistence (API `POST` / `DELETE`, or `localStorage`), edit dialog, all-day lane.
- Unit tests with Vitest + React Testing Library; keyboard navigation between events.
- "+3 more" collapsing when a group has too many columns.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why do touching events (10–11 and 11–12) not count as overlapping?

Because a meeting ending at 11:00 frees you at 11:00. Using `a.start < b.end && b.start < a.end` (strict) means they share no time. If we used `<=`, back-to-back meetings would shrink to half width for no reason.

Q2. What if events can cross midnight or span several days?

Split them at the day boundary at normalization time: a 23:00–01:00 event becomes 23:00–24:00 on day 1 and 00:00–01:00 on day 2. Each day still gets plain same-day intervals, so `layoutEvents` doesn't change.

Q3. What about time zones?

Store ISO timestamps on the server, convert to the viewer's zone once at the API boundary, then into minutes. The layout and rendering never see a zone. DST days have 23 or 25 hours, which a real app handles by computing minutes from the zoned day start.

State design

Q4. Why not store `col`, `cols` and `width` on each event in state?

They depend on *other* events. Adding one event can change the columns of five others. Storing them means syncing them by hand on every add/delete/move. Deriving them with `useMemo(layoutEvents(events))` is always consistent.

Q5. Why minutes instead of `Date` objects or "HH:MM" strings?

Numbers compare with `<`, subtract to a duration, and multiply to pixels. Strings need parsing everywhere; `Date`s bring time-zone surprises and are heavier. Convert once at the edges.

Q6. Who should own the events state, and why not context or Redux?

`App`, the closest common parent of the form and the grid. Two levels of props don't justify context. Context or a store makes sense once week view, sidebar and dialogs all read the same events.

Hard parts: the layout algorithm

Q7. Why do you sort by start, and why tie-break by longer duration first?

The sweep relies on start order: once an event starts after `groupEnd`, no earlier event can reach it. On equal starts, placing the longer event first puts it in the left column, which looks like Google Calendar and gives shorter events room to span.

Q8. What is the time complexity?

Sort `O(n log n)`. Placing each event scans the group's columns: `O(n·k)` where `k` is the max columns (small). Span is `O(g²)` per group. In practice, effectively `O(n log n)`. For thousands of overlapping events, keep free columns in a min-heap by end time to make placement `O(log k)`.

Q9. Why is first-fit column assignment optimal?

It is interval partitioning: processing intervals by start time and reusing any free column needs exactly as many columns as the maximum number of intervals overlapping at one instant, which is a lower bound. So no layout can use fewer columns.

Q10. Why not just compute "how many events overlap this one" and use that as the width?

It breaks on chains. With A 9–10, B 9:30–10:30, C 10–11: A overlaps 1, C overlaps 1, B overlaps 2. If A and C are 1/2 width but B is 1/3 width, positions collide. All events in a connected group must agree on one column count, hence clusters.

Q11. Why is span `O(g²)` acceptable?

`g` is the size of one overlap group, usually 2–5 events. Even 50 events is 2,500 checks, microseconds. Optimizing it would make the code harder to read for no user-visible benefit.

Implementation steps

Q12. Why absolute positioning instead of CSS Grid or flexbox?

Events start at arbitrary minutes and overlap; flex can't overlap items, and Grid would need 1440 rows or rounding. Absolute positioning inside a relative column gives exact pixel placement from simple numbers.

Q13. Why `useMemo` for the layout? Is it premature?

It's cheap insurance. Form drafts live inside `EventForm`, so typing doesn't re-render `App` today. But any future `App` state (selected event, a dialog, the current-time tick) re-renders `DayView`, and without `useMemo` the layout would rerun each time. It's the one non-trivial computation, so it's the one thing worth memoizing.

Q14. How would you add drag-to-move?

`onPointerDown` on the block stores the pointer Y and original start; `onPointerMove` computes `deltaMinutes = deltaY / pxPerMin`, snaps to 15, and updates a temporary `dragging` state; `onPointerUp` commits via `setEvents` (and the API, optimistic with rollback on failure). The layout recalculates automatically.

Q15. How do you scroll to 8 AM on load, and why a ref instead of `scrollIntoView`?

`scrollRef.current.scrollTop = 8 * HOUR_HEIGHT` in an effect after mount. It scrolls only the grid container, not the whole page, and needs no extra DOM node to target.

Performance and "what if"

Q16. What if a day has 1,000 events?

The algorithm is still fast. Rendering 1,000 absolutely positioned nodes is fine too, but a single slot with 50 overlaps becomes unreadable: cap columns (e.g. 4) and show a "+N more" chip that opens a list. For a week view, virtualize day columns outside the viewport.

Q17. What if the API is slow or fails?

Show a loading state immediately (we do), then an error with Retry. With a real API: abort on unmount/date change (`AbortController`) so a slow old response can't overwrite a newer day, and cache days already loaded.

Q18. What if the user switches days quickly?

Race condition: day 1's response may arrive after day 2's. Track the requested date (or use `AbortController`) and ignore responses whose date isn't the current one.

Trade-offs and "why not X?"

Q19. Why not give each event its *own* maximal width instead of a shared column count?

Fully maximal widths can make neighbours' edges misalign and the result hard to predict. Equal columns plus right-expansion (span) is what major calendars do: readable, stable, and simple to explain.

Q20. How would you make the calendar accessible beyond aria-labels?

Expose events as a list in time order (visually hidden or as the reading order), make each event focusable with arrow-key navigation, announce add/delete via a live region, and make the form time inputs fully labelled (done). Colour shouldn't be the only signal.

Q21. How would you test this?

Unit-test `layoutEvents` heavily (pure function: chains, touches, reuse, span, empty). Component tests with RTL: renders, add validation errors, delete removes. One visual / e2e test for positions. Most bugs live in the algorithm, so that's where most tests go.